

A Brief Overview of Microservices



The microservices architecture is a design style for software applications, which depicts them as a suite of independent deployable services. Microservices is about decomposing a complex system into more manageable, smaller units of work that can be developed and deployed independently. While microservices is not a panacea for all software problems, it can be leveraged to produce better applications.

Microservices is the new buzz word but it is often misunderstood. Before talking about microservices, we need to look into the past and see how it evolved over a period of time. I started out as a mainframe COBOL programmer. Back in those days, we used to write the driver or the main COBOL program, which in turn, would call programs that would call yet more programs. The called programs were either statically or dynamically called in the calling program. The entire set of programs used to be compiled through the mainframe compiler and then edit jobs were linked. If the programs were statically called, any change would mean recompilation, and link editing of the entire set of programs and redeployment.

A similar approach existed in the Java world as well, where the entire application used to be deployed as an EAR file or a WAR file. These single binary deployments were decomposed into data access, business process and presentation layers which we called N-tier architecture. The layers allowed the separation of concerns, but the application still used to be deployed as a single monolith. For any new change or enhancement, it would take days to do the impact analysis, the testing and finally, the deployment into production. The coupling of code in monoliths used to make it difficult to do an impact analysis, and changes were not localised but spread across the applications.